exactly like you did in chapter 7 (section 7.4.2) when securing the connection to an OAuth2 authorization server, as shown in listing 12.1. First, obtain an `SSLContext` for DTLS by calling `SSLContext.getInstance("DTLS")`. This will return a context that allows DTLS connections with any supported protocol version (DTLS 1.0 and DTLS 1.2 in Java 11). You can then load the certificates of trusted certificate authorities (CAs) and use this to initialize a `TrustManagerFactory`, just as you've done in previous chapters. The `TrustManagerFactory` will be used by Java to determine if the server's certificate is trusted. In this, case you can use the `as.example.com.ca.p12` file that you created in chapter 7 containing the mkcert CA certificate. Finally, you can initialize the `SSLContext` object, passing in the trust managers from the factory, using the `SSLContext.init()` method. This method takes three arguments:

- An array of `KeyManager` objects, which are used if performing client certificate authentication (covered in chapter 11). Because this example doesn't use client certificates you can leave this null.
- The array of `TrustManager` objects obtained from the `TrustManagerFactory`.
- An optional `SecureRandom` object to use when generating random key material and other data during the TLS handshake. You can leave this `null` in most cases to let Java choose a sensible default.

Create a new file named DtlsClient.java in the src/main/com/manning/apisecurityinaction folder and type in the contents of the listing.